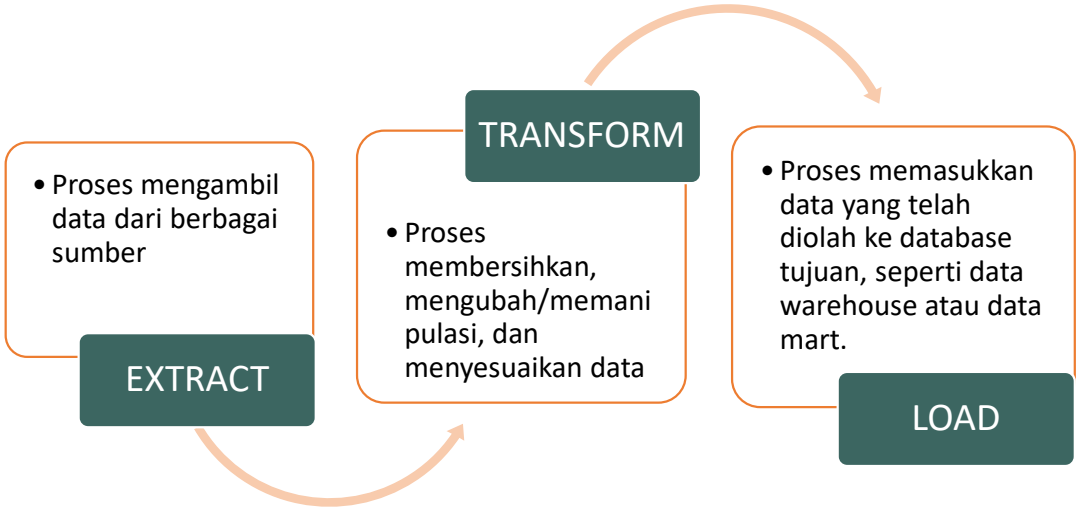
Day 25 - Extract, Transform, Load

by Dita Aulia – DABI 22

Apa itu ETL dan Mengapa Penting?



ETL adalah serangkaian proses mulai dari mengambil data dari berbagai sumber, kemudian mengolah data tersebut seperti membersihkan/mengubah/menyesuaikan, dan terakhir menyimpan data yang telah diolah ke dalam data warehouse atau data mart.

Mengapa ETL penting?

ETL penting karena;

- Mengintegrasikan data dari berbagai sumber.
- Membersihkan data yang tidak konsisten.
- Mengurangi duplikasi data.
- Meningkatkan kualitas data.
- Memudahkan analisis dan pembuatan laporan.
- Mempercepat pengambilan keputusan.

Bagaimana Proses Extract, Transform, dan Load Dilakukan?

EXTRACT
Proses mengambil data dari sumber.

```
SELECT *
FROM
public.ecommerce_tra
nsaction;
```

TRANSFORM
Proses mengolah data, bisa berupa menghilangkan duplikasi, mengubah format waktu.
Menghilangkan duplikasi disini digunakan untuk menyimpan data pada dimensi produk misalnya;

```
GROUP BY
src.product_id,
src.product_name,
src.product_category,
src.product_price
```

Sehingga yang akan tersimpan hanya satu.

LOAD
Data yang telah diolah pada proses transform kemudian disimpan pada data warehouse

```
INSERT INTO
dwh.dim_ecommerce_product
INSERT INTO
dwh.dim_ecommerce_store
INSERT INTO
dwh.dim_ecommerce_user
INSERT INTO
dwh.fact_ecommerce_transactio
n
```

Bagaimana data ditransformasikan, dimuat dan di otomatisasi dalam tahap pembuatan datamart?

Transformasi dilakukan setelah data berhasil diambil dari sumber, kemudian dimasukkan ke dalam staging, stg.stg_ecommerce_transaction

Setelah itu data dibersihkan dan disusun ulang agar sesuai

```
CREATE TABLE IF NOT EXISTS dwh.dim_ecommerce_product (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(255),
    product_category VARCHAR(50),
    product_price FLOAT4,
    last_update TIMESTAMP DEFAULT CURRENT_TIMESTAMP);
```

Dilakukan juga untuk setiap dimensi yang digunakan yakni dimensi store, dimensi users, dan fact sales.

Bagaimana data ditransformasikan, dimuat dan di otomatisasi dalam tahap pembuatan datamart?

TRUNCATE TABLE dwh.dim_ecommerce_product;
Menghapus data yang ada didalam kolom tabel yang sebelumnya dibuat.
Kemudian mengisi tabel dengan data terbaru: (Load)

```
INSERT INTO dwh.dim_ecommerce_product (product_id, product_name, product_category,
product_price, last_update)
SELECT src.product_id, src.product_name, src.product_category, src.product_price,
CURRENT_TIMESTAMP
FROM stg.stg_ecommerce_transaction AS src
GROUP BY src.product_id, src.product_name, src.product_category, src.product_price;
```

Groupby digunakan untuk mengambil satu data untuk satu produk.

Bagaimana data ditransformasikan, dimuat dan di otomatisasi dalam tahap pembuatan datamart?

Setelah itu, untuk menggabungkan semua data atau menggabungkan tabel fact_sales dengan seluruh table dimensi:

```
CREATE OR REPLACE VIEW dm.vw_dm_cube_ecommerce_transaction AS
  SELECT
    fs.sale_id,
    fs.transaction_date,
    fs.transaction_time,
    fs.store_id,
    ds.store_name,
    ds.store_city,
    fs.user_id,
    du.user_name,
    du.user_city,
    fs.product_id,
    dp.product_name,
    dp.product_category,
    dp.product_price,
    fs.quantity,
    fs.total_price,
    fs.payment_method,
    fs.transaction_status,
    fs.shipping_method,
    fs.last_update
  FROM
    dwh.fact_ecommerce_transaction fs
  LEFT JOIN
    dwh.dim_ecommerce_store ds ON fs.store_id = ds.store_id
  LEFT JOIN
    dwh.dim_ecommerce_user du ON fs.user_id = du.user_id
  LEFT JOIN
    dwh.dim_ecommerce_product dp ON fs.product_id =
dp.product_id;
```

Bagaimana data ditransformasikan, dimuat dan di otomatisasi dalam tahap pembuatan datamart?

Serangkaian proses otomatisasi dilakukan dengan menggunakan stored procedure:

```
CREATE OR REPLACE PROCEDURE dwh.generate_ecommerce_transaction()
 LANGUAGE plpgsql
AS $procedure$
BEGIN
……..
……..
END;
$procedure$
;
```

Pada stored procedure disini sudah melingkupi proses extract, transform, dan load.

untuk menjalankan seluruh proses diatas cukup dengan menggunakan:

```
CALL dwh.generate_ecommerce_transaction();
```

Screenshot query

```sql
CREATE OR REPLACE PROCEDURE dwh.generate_ecommerce_transaction()
    LANGUAGE plpgsql
AS $procedure$
BEGIN
    -- Step 1: Load Data into Staging (stg)
    CREATE TABLE IF NOT EXISTS stg.stg_ecommerce_transaction AS
    SELECT *, CURRENT_TIMESTAMP AS last_update FROM public.ecommerce_transaction;
    TRUNCATE TABLE stg.stg_ecommerce_transaction;
    INSERT INTO stg.stg_ecommerce_transaction
    SELECT *, CURRENT_TIMESTAMP AS last_update FROM public.ecommerce_transaction;
```

```sql
-- Step 2: Load Data into Dim_Product
CREATE TABLE IF NOT EXISTS dwh.dim_ecommerce_product (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(255),
    product_category VARCHAR(50),
    product_price FLOAT4,
    last_update TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
TRUNCATE TABLE dwh.dim_ecommerce_product;
INSERT INTO dwh.dim_ecommerce_product (product_id, product_name, product_category, product_price, last_update)
SELECT src.product_id, src.product_name, src.product_category, src.product_price, CURRENT_TIMESTAMP
FROM stg.stg_ecommerce_transaction AS src
GROUP BY src.product_id, src.product_name, src.product_category, src.product_price;
```

```sql
-- Step 5: Load Data into Fact_Sales
CREATE TABLE IF NOT EXISTS dwh.fact_ecommerce_transaction (
    sale_id INT PRIMARY KEY,
    transaction_date DATE,
    transaction_time TIME,
    store_id INT,
    user_id INT,
    product_id INT,
    quantity INT,
    total_price FLOAT,
    payment_method VARCHAR(50),
    transaction_status VARCHAR(50),
    shipping_method VARCHAR(50),
    last_update TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
INSERT INTO dwh.fact_ecommerce_transaction (sale_id, transaction_date, transaction_time, store_id, user_id, product_id, quantity, total_price,
SELECT
    src.transaction_id AS sale_id,
    src.transaction_time::DATE AS transaction_date,
    src.transaction_time::TIME AS transaction_time,
    src.store_id,
    src.user_id,
    src.product_id,
    src.quantity,
```

Screenshot query

```
    src.product_id,
    src.quantity,
    src.total_price,
    src.payment_method,
    src.transaction_status,
    src.shipping_method,
    CURRENT_TIMESTAMP AS last_update
FROM stg.stg_ecommerce_transaction AS src
WHERE NOT EXISTS (
    SELECT 1 FROM dwh.fact_ecommerce_transaction AS fact
    WHERE fact.sale_id = src.transaction_id
);
```

```
-- Step 6: Create Data Mart View (vw_dm_cube_ecommerce_transaction)
CREATE OR REPLACE VIEW dm.vw_dm_cube_ecommerce_transaction AS
SELECT
    fs.sale_id,
    fs.transaction_date,
    fs.transaction_time,
    fs.store_id,
    ds.store_name,
    ds.store_city,
    fs.user_id,
    du.user_name,
    du.user_city,
    fs.product_id,
    dp.product_name,
    dp.product_category,
    dp.product_price,
    fs.quantity,
    fs.total_price,
    fs.payment_method,
    fs.transaction_status,
    fs.shipping_method,
    fs.last update
FROM
    dwh.fact_ecommerce_transaction fs
LEFT JOIN
    dwh.dim_ecommerce_store ds ON fs.store_id = ds.store_id
LEFT JOIN
    dwh.dim_ecommerce_user du ON fs.user_id = du.user_id
LEFT JOIN
    dwh.dim_ecommerce_product dp ON fs.product_id = dp.product_id;
```

Screenshot query

Bagaimana implementasi partitioning meningkatkan efisiensi pengolahan data?

```
DO $$
DECLARE d DATE := '2025-01-01';
BEGIN
    WHILE d <= '2025-03-28' LOOP
        EXECUTE format(
            'CREATE TABLE IF NOT EXISTS dm.dm_cube_ecommerce_transaction_%s
            PARTITION OF dm.dm_cube_ecommerce_transaction
            FOR VALUES FROM (%L) TO (%L);',
            to_char(d, 'YYYYMMDD'), d, d + INTERVAL '1 day'
        );
        d := d + INTERVAL '1 day';
    END LOOP;
END $$;
```

Tabel dm.dm_cube_ecommerce_transaction dibagi menjadi beberapa partisi berdasarkan rentang tanggal (transaction_date). Dengan kata lain, data transaksi tidak disimpan dalam satu tabel besar, tetapi dipisahkan ke dalam tabel-tabel yang lebih kecil sesuai tanggal transaksi. Disini partisi dibuat secara otomatis menggunakan perulangan.

Manfaat partitioning terhadap efisiensi pengolahan data yakni dapat mempercepat proses query dimana database hanya membaca partisi yang dibutuhkan saja, sehingga waktu pencarian data menjadi lebih cepat. Kemudian karena data yang dibaca lebih sedikit, maka penggunaan CPU, memori menjadi lebih efisien.

Kesimpulan dan manfaat dari proses ETL yang telah dilakukan

Melalui stored procedure yang telah dibuat, maka berhasil pula pipeline yang dibangun, dengan manfaat antara lain:

- Seluruh proses mulai dari extract, transform, load dengan membuat dimensi dan fact, dirangkum dalam satu prosedur (otomatisasi), sehingga mengurangi pekerjaan manual.
- Data tidak terlalu berat karena sudah dipisah-pisah dan menerapkan partition. Ketika saat digunakan pun tidak terlalu memakan waktu yang lama
- Data akurat, terstruktur dan selalu diperbarui, sehingga memudahkan untuk membantu manajemen dalam mengambil keputusan bisnis yang lebih cepat dan tepat.

Thank you

by Dita Aulia – DABI 22